

Handling Missing Data in ETL

Name: Nomit Bagaria

Assignment Name: Handling Missing Data in ETL

Mob.No: 9828255290

Email: bagarianomit@gmail.com

SECTION A – THEORETICAL QUESTIONS

Q1: What are the most common reasons for missing data in ETL pipelines?

Answer

Missing data is a common issue in ETL (Extract, Transform, Load) pipelines and can occur for several reasons during data collection, storage, or integration.

Common Reasons for Missing Data

1. Human Data Entry Errors

Users may forget to enter required information or accidentally leave fields blank.

2. System or Software Errors

Technical issues during data collection or transfer can result in missing values.

3. Data Integration Issues

When combining data from multiple sources, some fields may not exist in every source.

4. Sensor or Device Failure

IoT devices or sensors may temporarily stop recording data.

5. Data Corruption

Files may become corrupted during storage or transmission.

6. Optional Fields

Some information may not be mandatory during data collection.

Q2: Why is blindly deleting rows with missing values considered a bad practice in ETL?

Answer

Blindly deleting rows with missing values can lead to the loss of valuable information and reduce the quality of analysis.

Reasons

- Reduces the size of the dataset.
- May remove important business information.
- Can introduce bias if missing values are not random.
- Decreases the performance of machine learning models due to less training data.
- Some missing values can be recovered through imputation instead of deletion.

Example

If customer income is missing for premium customers only, deleting those rows may produce inaccurate business insights. Rows should only be deleted after carefully evaluating the amount and pattern of missing data. Imputation or other techniques are often better alternatives.

Q3: Explain the difference between:

• Listwise Deletion

• Column Deletion

Also mention one scenario where each is appropriate.

Answer

Listwise Deletion	Column Deletion
Removes entire rows containing missing values.	Removes an entire column containing excessive missing values.
Suitable when only a few rows contain missing data.	Suitable when a column has a very high percentage of missing values.

Preserves all remaining columns.

Preserves all remaining rows.

May reduce the number of records.

May remove an important feature.

Scenario for Listwise Deletion

A dataset of 10,000 records contains only 15 rows with missing values.

Removing these few rows has minimal impact.

Scenario for Column Deletion

A column contains 95% missing values and provides little useful information.

Deleting the column is more practical than attempting imputation.

Q4: Why is median imputation preferred over mean imputation for skewed data such as income?

Answer

Median imputation is preferred because it is less affected by extreme values (outliers).

Mean Imputation

- Uses the average value.
- Strongly influenced by very high or very low values.

Median Imputation

- Uses the middle value after sorting.
- Remains stable even when outliers exist.

Example

Income values: 25000, 28000, 30000, 35000, 500000

Mean: 123600

Median: 30000

The median better represents the typical income.

So, for skewed data such as salaries or income, median imputation provides more realistic estimates because it is not heavily influenced by outliers.

Q5: What is forward fill and in what type of dataset is it most useful?

Answer

Forward Fill (ffill) is a missing value imputation technique that replaces missing values with the last valid observation.

Forward fill is most suitable for sequential or time-series datasets where previous observations remain valid until new values become available.

Example

Day	Sales
Monday	100
Tuesday	NaN
Wednesday	NaN
Thursday	150

After Forward Fill

Day	Sales
Monday	100
Tuesday	100
Wednesday	100
Thursday	150

Most Useful For

- Time-series data
- Sensor readings
- Stock prices
- Weather data
- Daily sales records

Q6: Why should flagging missing values be done before imputation in an ETL workflow?

Answer

Flagging missing values creates an indicator showing which records originally contained missing data before those values are replaced.

Benefits

- Preserves information about missingness.
- Helps analysts identify patterns in missing data.
- Improves machine learning by allowing models to learn whether missingness itself is important.
- Maintains transparency during data preprocessing.
- Supports auditing and debugging of ETL pipelines.

Example

Income	Income_Missing_Flag
65000	0
NaN	1
72000	0

After imputation, the flag still indicates which value was originally missing. So, creating missing-value flags before imputation preserves valuable information and improves both data analysis and model performance.

Q7: Consider a scenario where income is missing for many customers. How can this missingness itself provide business insights?

Answer

Missing income values may reveal important business patterns instead of simply indicating incomplete data.

Possible Business Insights

- Customers may intentionally avoid sharing income due to privacy concerns.
- Missing income may be more common in specific regions or customer groups.
- Certain branches or sales teams may not consistently collect income information.
- High-income or low-income customers may be more likely to leave the field blank.
- Missing income may indicate weaknesses in the customer onboarding process.

Example

If 70% of customers from one region have missing income values, the company may need to improve data collection procedures in that region.

Conclusion

Missing values can themselves provide useful insights into customer behavior, operational processes, and data quality. Therefore, they should be analyzed before applying imputation or deletion.

Section B – Practical Questions

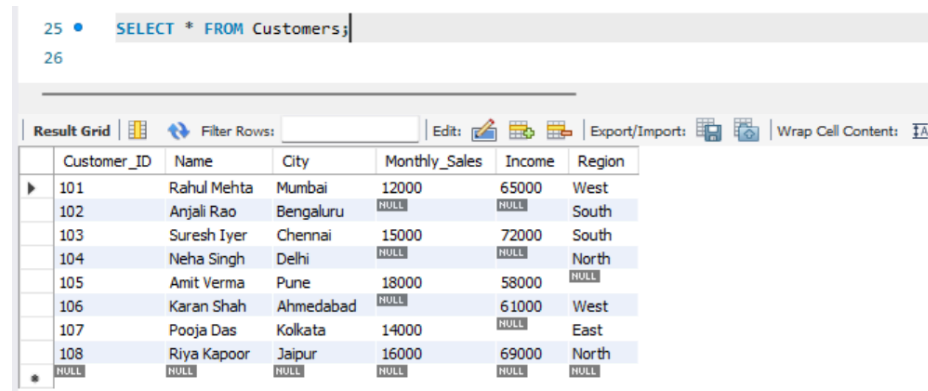
Database:

```
CREATE DATABASE ETL_Practice;  
USE ETL_Practice;
```

```
CREATE TABLE Customers (  
    Customer_ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    City VARCHAR(50),  
    Monthly_Sales INT,  
    Income INT,  
    Region VARCHAR(50)  
);
```

```
INSERT INTO Customers  
(Customer_ID, Name, City, Monthly_Sales, Income, Region)  
VALUES  
(101,'Rahul Mehta','Mumbai',12000,65000,'West'),  
(102,'Anjali Rao','Bengaluru',NULL,NULL,'South'),  
(103,'Suresh Iyer','Chennai',15000,72000,'South'),  
(104,'Neha Singh','Delhi',NULL,NULL,'North'),  
(105,'Amit Verma','Pune',18000,58000,NULL),  
(106,'Karan Shah','Ahmedabad',NULL,61000,'West'),  
(107,'Pooja Das','Kolkata',14000,NULL,'East'),  
(108,'Riya Kapoor','Jaipur',16000,69000,'North');
```

```
SELECT * FROM Customers;
```



25 • SELECT * FROM Customers;

26

Result Grid | Filter Rows: | Edit: | Export/Imports: | Wrap Cell Content: |

	Customer_ID	Name	City	Monthly_Sales	Income	Region
▶	101	Rahul Mehta	Mumbai	12000	65000	West
	102	Anjali Rao	Bengaluru	NULL	NULL	South
	103	Suresh Iyer	Chennai	15000	72000	South
	104	Neha Singh	Delhi	NULL	NULL	North
	105	Amit Verma	Pune	18000	58000	NULL
	106	Karan Shah	Ahmedabad	NULL	61000	West
	107	Pooja Das	Kolkata	14000	NULL	East
	108	Riya Kapoor	Jaipur	16000	69000	North
*	NULL	NULL	NULL	NULL	NULL	NULL

Q8. Listwise Deletion

Remove all rows where **Region** is missing.

Tasks:

1. Identify affected rows.
2. Show the dataset after deletion.
3. Mention how many records were lost

Answer:.

```
SELECT * FROM customers  
WHERE Region IS NULL;
```

```
27 • SELECT * FROM customers
28 WHERE Region IS NULL;
```

Result Grid	Filter Rows:	Edit:	Export/Im		
Customer_ID	Name	City	Monthly_Sales	Income	Region
▶ 105	Amit Verma	Pune	18000	58000	NULL
* NULL	NULL	NULL	NULL	NULL	NULL

```
DELETE FROM customers  
WHERE Region IS NULL;
```

```
SELECT * FROM customers;
```

```
30 • DELETE FROM customers
31 WHERE Region IS NULL;
32
33 • SELECT * FROM customers;
```

Customer_ID	Name	City	Monthly_Sales	Income	Region
101	Rahul Mehta	Mumbai	12000	65000	West
102	Anjali Rao	Bengaluru	NULL	NULL	South
103	Suresh Iyer	Chennai	15000	72000	South
104	Neha Singh	Delhi	NULL	NULL	North
106	Karan Shah	Ahmedabad	NULL	61000	West
107	Pooja Das	Kolkata	14000	NULL	East
108	Riya Kapoor	Jaipur	16000	69000	North
NULL	NULL	NULL	NULL	NULL	NULL

```
SELECT COUNT(*) AS Remaining_Records  
FROM customers;
```

```
35 • SELECT COUNT(*) AS Remaining_Records
36 FROM customers;
37
38
```

Result Grid		 Filter Rows:	Exports: 	Wrap Cell Content: 
Remaining_Records				
7				

CONCLUSION :

One record had a missing value in the **Region** column. Using listwise deletion, that row was removed from the dataset. The total number of records decreased from **8 to 7**, meaning **1 record was lost**.

Q9. Imputation

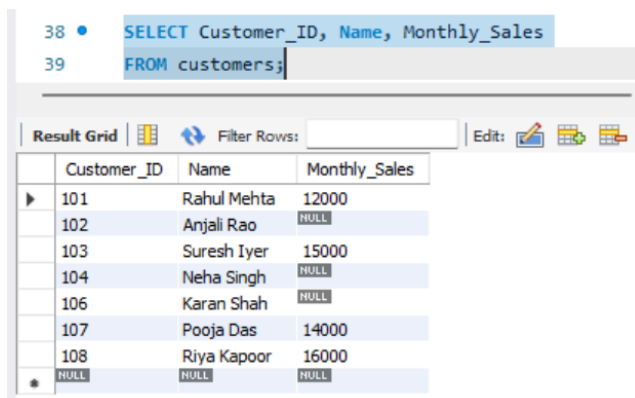
Handle missing values in **Monthly_Sales** using **Forward Fill**.

Tasks

1. Apply forward fill.
2. Show before vs after values.
3. Explain why forward fill is suitable here.

Answer:

```
SELECT Customer_ID, Name, Monthly_Sales
FROM customers;
```



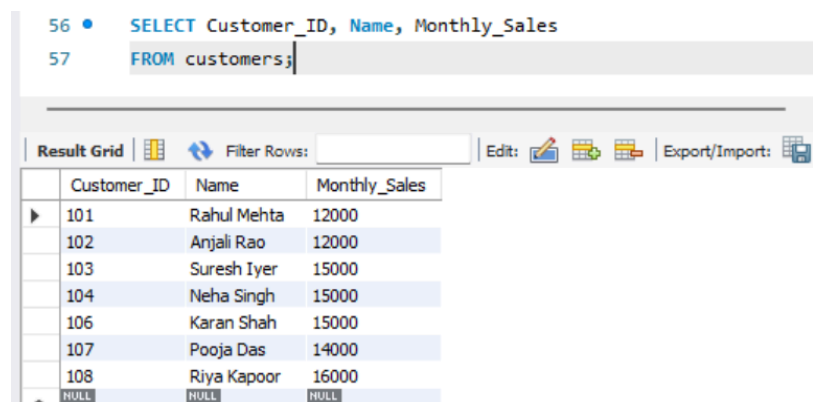
The screenshot shows a SQL query editor with the following code:

```
38 • SELECT Customer_ID, Name, Monthly_Sales
39 FROM customers;
```

Below the query, a 'Result Grid' displays the data from the 'customers' table. The table has three columns: Customer_ID, Name, and Monthly_Sales. The data is as follows:

Customer_ID	Name	Monthly_Sales
101	Rahul Mehta	12000
102	Anjali Rao	NULL
103	Suresh Iyer	15000
104	Neha Singh	NULL
106	Karan Shah	NULL
107	Pooja Das	14000
108	Riya Kapoor	16000
NULL	NULL	NULL

```
UPDATE customers c
SET Monthly_Sales = (
    SELECT Monthly_Sales
    FROM (
        SELECT Customer_ID, Monthly_Sales
        FROM customers
    ) AS temp
    WHERE temp.Customer_ID < c.Customer_ID
    AND temp.Monthly_Sales IS NOT NULL
    ORDER BY temp.Customer_ID DESC
    LIMIT 1
)
WHERE Monthly_Sales IS NULL;
```



The screenshot shows a SQL query editor with the following code:

```
56 • SELECT Customer_ID, Name, Monthly_Sales
57 FROM customers;
```

Below the query, a 'Result Grid' displays the data from the 'customers' table after the forward fill operation. The table has three columns: Customer_ID, Name, and Monthly_Sales. The data is as follows:

Customer_ID	Name	Monthly_Sales
101	Rahul Mehta	12000
102	Anjali Rao	12000
103	Suresh Iyer	15000
104	Neha Singh	15000
106	Karan Shah	15000
107	Pooja Das	14000
108	Riya Kapoor	16000
NULL	NULL	NULL

```
SELECT Customer_ID, Name, Monthly_Sales  
FROM customers;
```

Explanation :

Forward Fill replaces missing values with the most recent available value from previous records. It is commonly used in sequential or time-ordered datasets where nearby observations are expected to have similar values. In this dataset, the missing Monthly_Sales values were filled using the previous customer's available sales value.

Q10. Flagging Missing Data

Create a **flag column** for missing Income.

Tasks

1. Create Income_Missing_Flag (0 = Present, 1 = Missing)
2. Show updated dataset.
3. Count customers with missing income.

Answer:

```
ALTER TABLE customers
```

```
ADD Income_Missing_Flag INT;
```

```
UPDATE customers
```

```
SET Income_Missing_Flag =
```

```
CASE
```

```
    WHEN Income IS NULL THEN 1
```

```
    ELSE 0
```

```
END;
```

```
SELECT Customer_ID, Name, Income, Income_Missing_Flag  
FROM customers;
```

```
69 • SELECT Customer_ID, Name, Income, Income_Missing_Flag  
70 FROM customers;  
71
```

Customer_ID	Name	Income	Income_Missing_Flag
101	Rahul Mehta	65000	0
102	Anjali Rao	NULL	1
103	Suresh Iyer	72000	0
104	Neha Singh	NULL	1
106	Karan Shah	61000	0
107	Pooja Das	NULL	1
108	Riya Kapoor	69000	0
•	NULL	NULL	NULL

```
SELECT COUNT(*) AS Missing_Income_Customers  
FROM customers  
WHERE Income_Missing_Flag = 1;
```

```
72 • SELECT COUNT(*) AS Missing_Income_Customers  
73 FROM customers  
74 WHERE Income_Missing_Flag = 1;  
75
```

Missing_Income_Customers
3

Explanation :

An `Income_Missing_Flag` column was created to identify records with missing income values. Customers with available income were assigned 0, while customers with missing income were assigned 1. A total of 3 customers had missing income values. Flagging missing values preserves useful information about data quality before applying any imputation technique.